University of Dhaka

Department of Computer Science and Engineering

CSE 3216: Software Design Pattern Lab

Session: 2021-22

Software Design Patterns Implementation

Bus Recommendation System Refactoring Report

Team: Innova DU

Jannatul Ferdousi (08)
Md. Saif Mahamud (34)
Rezaunnabi Ruhan (58)
Mohammad Sajid Al Rafi Hasan (62)

Submitted To:

Md. Mahmudur Rahman, Assistant Professor, Dept. of CSE, University of Dhaka
Md. Fahim Arefin, Lecturer, Dept. of CSE, University of Dhaka

Submission Date: December 27, 2025

Contents

1 Introduction **3**
1.1 Problem Definition & Context . . . 3
1.2 Motivation . . . 3
1.3 Core Features . . . 3
1.4 Tools, Technologies & Frameworks Used . . . 4
1.5 Individual Responsibilities . . . 4
1.5.1 Jannatul Ferdousi . . . 4
1.5.2 Md. Saif Ahmed . . . 6
1.5.3 Mohammad Sajid Al Rafi Hasan . . . 7
1.5.4 Rezaunnabi Ruhan . . . 8

2 System Overview **9**
2.1 System Architecture . . . 9
2.2 Use Case Diagrams . . . 9

3 Design Patterns Used **9**
3.1 Design Philosophy & Rationale . . . 9
3.2 Strategy Pattern - Distance Calculation . . . 10
3.2.1 Problem Addressed . . . 10
3.2.2 Why This Pattern? . . . 10
3.2.3 UML Diagram . . . 10
3.2.4 Implementation Snippet . . . 10
3.2.5 Benefits Achieved . . . 12
3.2.6 Limitations & Trade-offs . . . 12
3.3 Observer Pattern - State Management . . . 12
3.3.1 Problem Addressed . . . 12
3.3.2 Why This Pattern? . . . 12
3.3.3 UML Diagram . . . 13
3.3.4 Implementation Snippet . . . 13
3.3.5 Benefits Achieved . . . 14
3.3.6 Limitations & Trade-offs . . . 14
3.4 Decorator Pattern - Bus Result Enhancement . . . 15
3.4.1 Problem Addressed . . . 15
3.4.2 Why This Pattern? . . . 15
3.4.3 UML Diagram . . . 16
3.4.4 Implementation Snippet . . . 16
3.4.5 Benefits Achieved . . . 18
3.4.6 Limitations & Trade-offs . . . 18
3.5 Singleton Pattern - Database Client Management . . . 18
3.5.1 Problem Addressed . . . 18
3.5.2 Why This Pattern? . . . 18
3.5.3 UML Diagram . . . 19
3.5.4 Implementation Snippet . . . 19
3.5.5 Benefits Achieved . . . 21
3.5.6 Limitations & Trade-offs . . . 21

3.6 Builder Pattern - Bus Filtering . . . 21
3.6.1 Problem Addressed . . . 21
3.6.2 Why This Pattern? . . . 21
3.6.3 UML Diagram . . . 22
3.6.4 Implementation Snippet . . . 22
3.6.5 Benefits Achieved . . . 24
3.6.6 Limitations & Trade-offs . . . 24

4 System Implementation **24**
4.1 UI Screens . . . 24
4.2 Code Modules & Responsibilities . . . 24
4.3 File/Package Structure . . . 25
4.4 Database Schema . . . 26
4.5 Code Repository Link . . . 27
4.6 Video Demo Link . . . 27

5 Evaluation **27**
5.1 Evidence of Improvement . . . 27
5.1.1 Code Quality Metrics . . . 27
5.1.2 Performance Improvements . . . 27
5.2 Comparison With Non-Pattern Alternative . . . 27
5.3 Maintainability Assessment . . . 27

6 Conclusion **28**
6.1 Challenges & Solutions . . . 28
6.1.1 Challenge 1: Complex State Management . . . 28
6.1.2 Challenge 2: Multiple Distance Calculation Methods . . . 28
6.1.3 Challenge 3: Complex Bus Filtering . . . 28
6.1.4 Challenge 4: Result Enhancement . . . 28
6.2 Lessons Learned . . . 28
6.3 Future Improvements . . . 28
6.4 Reflection on Design Principles . . . 29

A UML Diagrams **29**
A.1 Complete System Architecture . . . 29

B Glossary **29**

C References **30**

1 Introduction

1.1 Problem Definition & Context

The Dhaka Bus Route Planning System addresses the critical challenge of efficient public transportation navigation in one of the world's most densely populated cities. With over 9 million residents relying on an extensive but complex bus network, finding optimal routes between any two locations presents significant computational and user experience challenges.

The system was initially developed as a basic route finder but required substantial refactoring to implement proper software design patterns, improve maintainability, and enhance performance. This project demonstrates the systematic application of multiple design patterns to transform a monolithic codebase into a well-structured, extensible application.

1.2 Motivation

The motivation for this refactoring project stems from several key factors:

- **Code Maintainability**: The original codebase suffered from tight coupling and scattered business logic, making it difficult to modify or extend.
- **Performance Optimization**: Distance calculations and route planning required optimization through strategic caching and algorithm selection.
- **Scalability**: The system needed to handle multiple distance calculation strategies and filtering mechanisms without code duplication.
- **Educational Value**: This project serves as a comprehensive demonstration of how design patterns solve real-world software engineering problems.

1.3 Core Features

The Bus Route Planning System provides the following key functionalities:

- **Threshold-Based Stop Discovery**: Find bus stops within configurable distances (100m to 5000m) from user locations
- **Multi-Strategy Distance Calculation**: OSRM integration with Haversine fallback for reliable distance computation
- **Advanced Bus Filtering**: Filter by AC/Non-AC status, coach types (Standard-/Express/Luxury), journey length, and walking distance
- **Journey Enhancement**: Automatic calculation of journey lengths, walking distances, and time estimates
- **Reactive State Management**: Real-time UI updates through observer pattern implementation
- **Accessibility Compliance**: WCAG AA compliant interface with full keyboard navigation

1.4 Tools, Technologies & Frameworks Used

- **Frontend**: Next.js 15, React 19, TypeScript, Tailwind CSS
- **State Management**: Zustand with custom Observer pattern implementation
- **Database**: Supabase (PostgreSQL) with custom migrations
- **Maps & Routing**: Leaflet, React-Leaflet, OSRM (Open Source Routing Machine)
- **UI Components**: shadcn/ui with Radix UI primitives
- **Development Tools**: ESLint, TypeScript compiler, tsx for script execution

1.5 Individual Responsibilities

1.5.1 Jannatul Ferdousi

Core System Architecture:

- Complete system architecture design and implementation
- Project structure organization and module separation
- Technology stack selection and integration
- Main database schema design and real world data insertion(buses, stops, routes, route_stops)
- API endpoint architecture and implementation

Bus Route and OSRM Integration:

- Primary development of bus route system (with assistance from Saif)
- OSRM server integration and configuration
- Distance calculation algorithms and optimization
- Route finding and path calculation logic
- Geographic data processing and coordinate systems

Design Pattern Implementations:

- **Strategy Pattern**: Complete implementation of distance calculation strategies
  - OSRMStrategy class with API integration and error handling
  - HaversineStrategy class with mathematical calculations
  - DistanceCalculator context class with automatic fallback logic
  - Strategy interface design and type definitions
- **Decorator Pattern**: Bus result enhancement system
  - BusResultDecorator abstract base class

  - JourneyLengthDecorator with distance calculations
  - WalkingDistanceDecorator with multi-point calculations
  - TimeEstimateDecorator with speed-based estimations
  - EnhancedBusResultFactory for composition

- **Builder Pattern**: Complex filtering system
  - BusFilterBuilder with fluent interface
  - Database-level query optimization
  - Client-side filtering for computed properties
  - Filter validation and error handling

User Management and Authentication System:

- Complete authentication system implementation
- Sign in and sign out functionality
- Forgotten password recovery system
- Role-based permission management
- User session handling and security

Bus Management System:

- Bus CRUD operations according to role and management interface
- Bus status tracking and updates
- Integration with route planning system

Community System:

- Community features architecture and frontend and backend implementation
- User interaction systems
- Content management and moderation
- Community-specific business logic

Bus Review System:

- Complete bus review and rating system
- Review submission and validation
- Rating aggregation and display
- Integration with bus information display

Service Layer Development:

- BusRouteService with complex route finding algorithms
- StopDiscoveryService with threshold-based filtering
- Distance calculation service integration
- Error handling and retry mechanisms
- Performance optimization and caching strategies

Testing Framework:

- Integration tests for API endpoints
- Mock implementations for external services
- Test utilities and helper functions

Documentation:

- Complete technical documentation
- Code comments and type definitions

1.5.2 Md. Saif Ahmed

Design Pattern Implementations:

- **Singleton Pattern**: Database client management and connection pooling
  - Supabase client singleton implementation
  - Connection management and optimization
  - Client instance lifecycle management
- **Observer Pattern**: State management system implementation
  - Observable base class with subscription management
  - RoutePlannerStore with complex business logic
  - React hook integration for UI synchronization
  - State update optimization and memory leak prevention

Bus Route System Support:

- Assisted Jannatul with bus route system development
- Frontend integration of OSRM functionality
- Route visualization and mapping components
- User interface for route selection and display

Database Operations:

- Real data insertion scripts and utilities

- Data migration and seeding operations
- Database performance optimization
- Data validation and integrity checks

React Component Development:

- Route planner page component structure
- Form handling and user input validation
- Map component integration with Leaflet
- Responsive design implementation
- Loading states and error handling UI

State Management Integration:

- useRoutePlanner hook implementation
- Component-store connection logic
- React state synchronization with Observer pattern
- Event handling for user interactions

Frontend Testing:

- React component testing with Testing Library
- User interaction testing
- Component integration tests

1.5.3 Mohammad Sajid Al Rafi Hasan

Community System Database:

- Community-specific database schema design
- Community tables creation and relationships
- Community data insertion and seeding
- Community-related migration scripts
- User community interaction data management

Attempted UI Improvements:

- Basic UI styling attempts for filter components
- Minor CSS adjustments for responsive design
- Attempted improvements to community interface

- Basic form validation enhancements
- Color scheme and layout adjustments

Learning and Support:

- Participated in code reviews and discussions
- Attempted to understand design pattern implementations
- Provided feedback on user interface design
- Assisted with documentation proofreading
- Contributed to team planning sessions

1.5.4 Rezaunnabi Ruhan

Attempted UI Improvements:

- Minor UI component styling modifications
- Attempted integration of map markers and visual elements
- Basic responsive design adjustments
- Color and typography improvement attempts
- User interface layout refinements

Database Support Activities:

- Basic database query testing
- Attempted data validation scripts
- Minor database utility functions
- Data consistency checking attempts

Support Activities:

- Participated in team meetings and planning sessions
- Attempted to understand system architecture
- Provided input on user requirements
- Assisted with basic testing scenarios
- Contributed to project documentation review

2 System Overview

2.1 System Architecture

The system follows a layered architecture with clear separation of concerns:

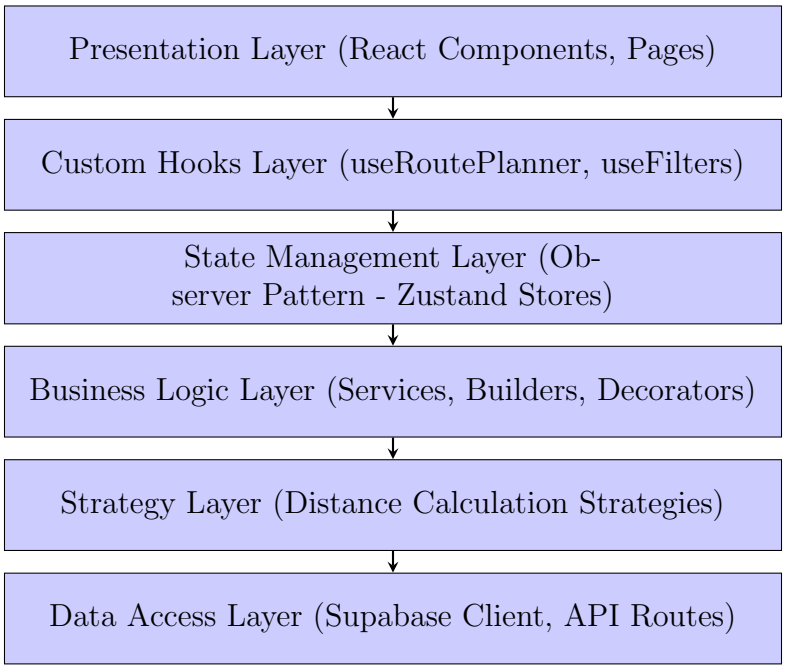


Figure 1: System Architecture Layers

2.2 Use Case Diagrams

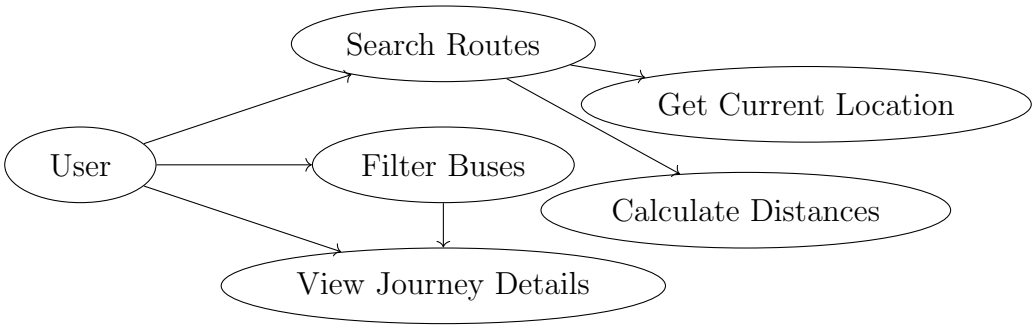


Figure 2: Primary Use Case Diagram

3 Design Patterns Used

3.1 Design Philosophy & Rationale

Our design pattern implementation follows key software engineering principles:

- **Single Responsibility Principle**: Each pattern addresses a specific concern
- **Open/Closed Principle**: Systems are open for extension, closed for modification

- **Dependency Inversion**: Depend on abstractions, not concretions
- **Composition over Inheritance**: Favor object composition for flexibility

3.2 Strategy Pattern - Distance Calculation

3.2.1 Problem Addressed

The application requires different distance calculation methods with varying trade-offs:

- OSRM provides accurate road-based distances but requires a running server
- Haversine formula works offline but calculates straight-line distances
- Future algorithms (Google Maps API, custom routing) need easy integration

3.2.2 Why This Pattern?

The Strategy Pattern enables:

- Runtime algorithm selection based on availability
- Automatic fallback when primary strategy fails
- Easy addition of new distance calculation methods
- Testable, independent algorithm implementations

3.2.3 UML Diagram

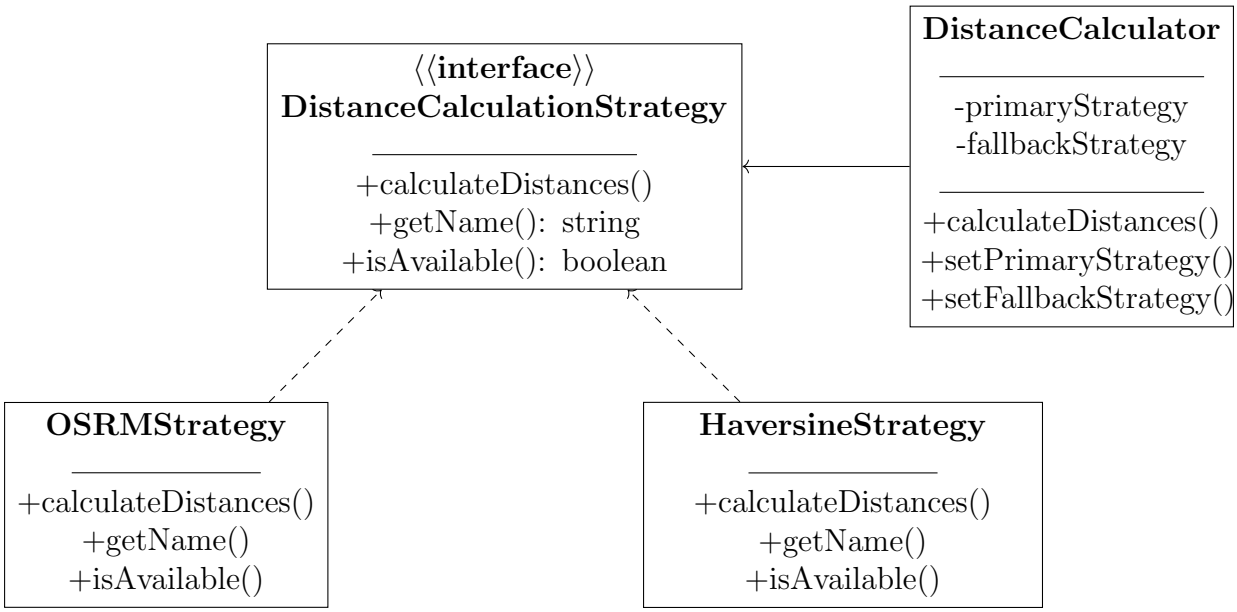


Figure 3: Strategy Pattern UML Diagram

3.2.4 Implementation Snippet

Listing 1: Strategy Pattern Implementation

```
// Strategy Interface
interface DistanceCalculationStrategy {
  calculateDistances(
    origins: Coordinate[],
    destinations: Coordinate[]
  ): Promise<DistanceResult[][]>

  getName(): string
  isAvailable(): boolean | Promise<boolean>
}

// Concrete Strategy - OSRM
class OSRMStrategy implements DistanceCalculationStrategy {
  async calculateDistances(origins, destinations) {
    const response = await
       fetch(`${this.osrmUrl}/table/v1/driving/...`)
    return this.parseOSRMResponse(response)
  }

  getName() { return "OSRM" }

  async isAvailable() {
    try {
      const response = await fetch(`${this.osrmUrl}/health`)
      return response.ok
    } catch {
      return false
    }
  }
}

// Context Class
class DistanceCalculator {
  async calculateDistances(origins, destinations, useFallback =
     true) {
    try {
      if (await this.primaryStrategy.isAvailable()) {
        return await
           this.primaryStrategy.calculateDistances(origins,
           destinations)
      }
    } catch (error) {
      console.warn(`Primary strategy failed: ${error.message}`)
    }

    if (useFallback) {
      return await
         this.fallbackStrategy.calculateDistances(origins,
         destinations)
    }

    throw new Error('No available distance calculation strategy')
  }
}
```

3.2.5 Benefits Achieved

- **Reliability**: Automatic fallback ensures system always works
- **Flexibility**: Easy to switch between algorithms at runtime
- **Maintainability**: Each algorithm is independently testable
- **Extensibility**: New strategies require only interface implementation

3.2.6 Limitations & Trade-offs

- Additional abstraction layer adds slight complexity
- Strategy switching has minimal runtime overhead
- Requires careful error handling for strategy failures

3.3 Observer Pattern - State Management

3.3.1 Problem Addressed

React components need to stay synchronized with complex application state changes:

- Route planning state spans multiple components
- State updates should trigger UI re-renders automatically
- Business logic should be decoupled from UI components

3.3.2 Why This Pattern?

The Observer Pattern provides:

- Loose coupling between state and UI components
- Automatic notification of state changes
- Centralized state management with distributed observation
- Easy testing of business logic separate from UI

3.3.3 UML Diagram

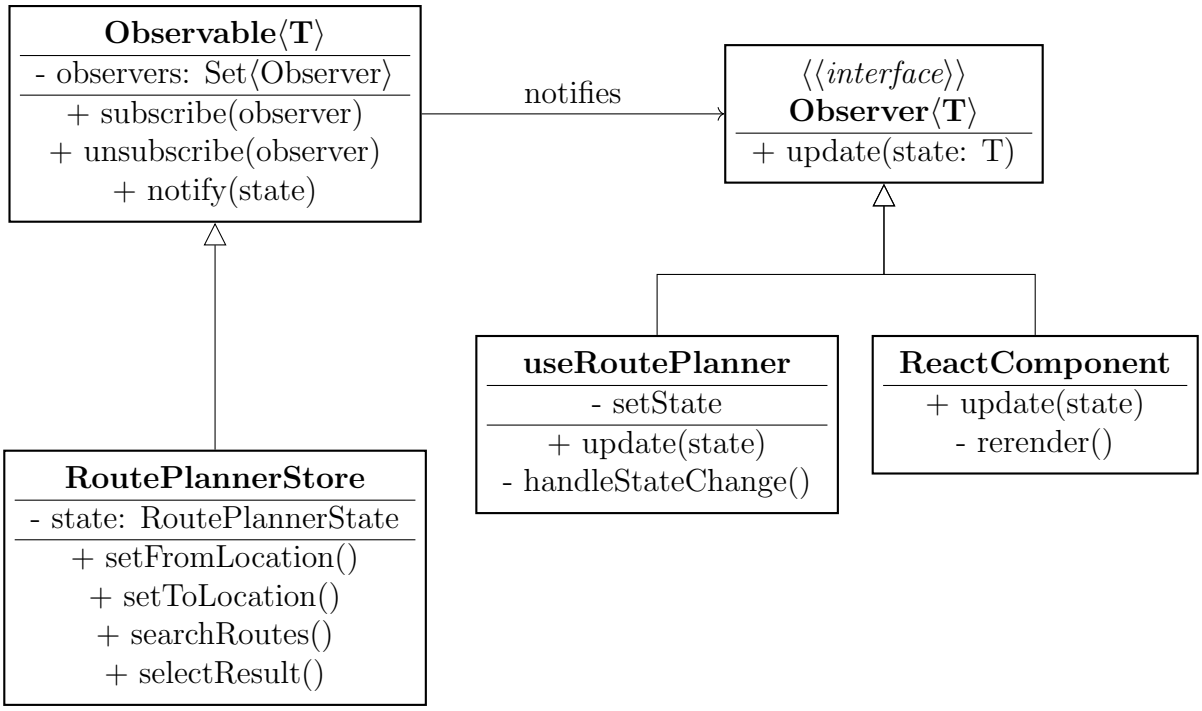


Figure 4: Observer Pattern UML Diagram

3.3.4 Implementation Snippet

Listing 2: Observer Pattern Implementation

```
// Observer Interface
interface Observer<T> {
  update(state: T): void
}

// Observable Base Class
class Observable<T> {
  private observers = new Set<Observer<T>>()

  subscribe(observer: Observer<T>) {
    this.observers.add(observer)
  }

  unsubscribe(observer: Observer<T>) {
    this.observers.delete(observer)
  }

  protected notify(state: T) {
    this.observers.forEach(observer => observer.update(state))
  }
}

// Concrete Observable - Route Planner Store
class RoutePlannerStore extends Observable<RoutePlannerState> {
  private state: RoutePlannerState = initialState
```

```

  setFromLocation(location: string) {
    this.setState({ fromLocation: location })
  }

  async searchRoutes() {
    this.setState({ isLoading: true })
    try {
      const results = await this.routeService.findRoutes(...)
      this.setState({ searchResults: results, isLoading: false })
    } catch (error) {
      this.setState({ error: error.message, isLoading: false })
    }
  }

  private setState(updates: Partial<RoutePlannerState>) {
    this.state = { ...this.state, ...updates }
    this.notify(this.state)
  }
}

// React Hook as Observer
function useRoutePlanner() {
  const [state, setState] = useState(routePlannerStore.getState())

  useEffect(() => {
    const observer = { update: setState }
    routePlannerStore.subscribe(observer)
    return () => routePlannerStore.unsubscribe(observer)
  }, [])

  return { state, actions: routePlannerStore }
}
```

3.3.5 Benefits Achieved

- **Decoupling**: UI components don't directly depend on business logic
- **Consistency**: All observers receive the same state updates
- **Testability**: Store logic can be tested independently
- **Scalability**: Easy to add new observers without modifying existing code

3.3.6 Limitations & Trade-offs

- Memory leaks possible if observers aren't properly unsubscribed
- All observers are notified on every state change (can be optimized)
- Debugging can be complex with many observers

3.4 Decorator Pattern - Bus Result Enhancement

3.4.1 Problem Addressed

Bus results from the database need additional computed properties:

- Journey length calculations between stops
- Walking distance computations to/from stops
- Time estimates for journey and walking
- Total distance and time calculations

3.4.2 Why This Pattern?

The Decorator Pattern enables:

- Adding functionality without modifying base objects
- Composable enhancements that can be applied selectively
- Separation of concerns between data and computed properties
- Easy testing of individual enhancement logic

3.4.3 UML Diagram

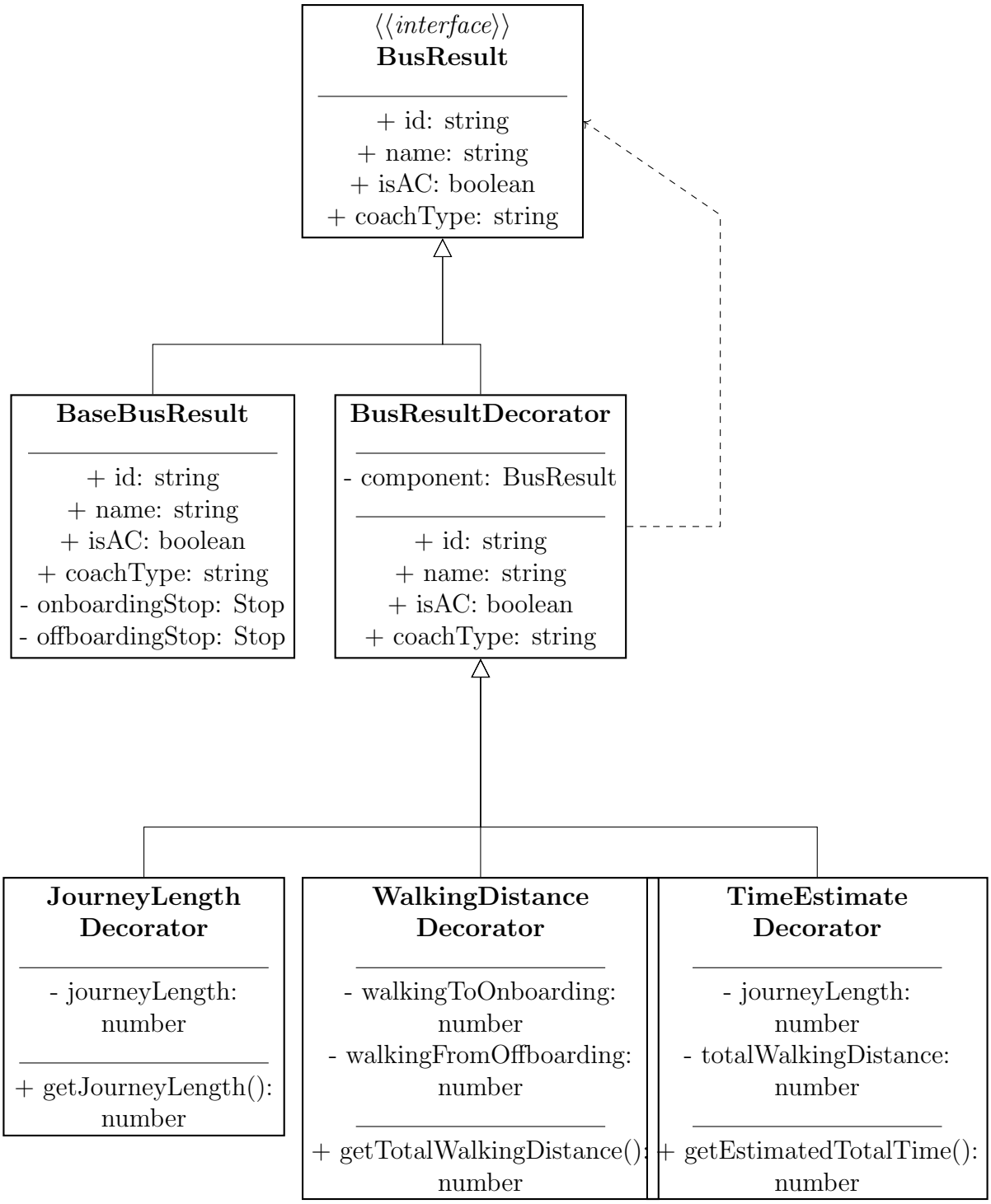


Figure 5: Decorator Pattern UML Diagram

3.4.4 Implementation Snippet

Listing 3: Decorator Pattern Implementation

```
// Base Decorator
```

```
abstract class BusResultDecorator implements BusResult {
  constructor(protected busResult: BusResult) {}

  get id() { return this.busResult.id }
  get name() { return this.busResult.name }
  get isAC() { return this.busResult.isAC }
  get coachType() { return this.busResult.coachType }
}

// Concrete Decorator - Journey Length
class JourneyLengthDecorator extends BusResultDecorator {
  constructor(busResult: BusResult, private journeyLength: number) {
    super(busResult)
  }

  getJourneyLength(): number {
    return this.journeyLength
  }
}

// Concrete Decorator - Walking Distance
class WalkingDistanceDecorator extends BusResultDecorator {
  constructor(
    busResult: BusResult,
    private walkingToOnboarding: number,
    private walkingFromOffboarding: number
  ) {
    super(busResult)
  }

  getTotalWalkingDistance(): number {
    return this.walkingToOnboarding + this.walkingFromOffboarding
  }
}

// Factory for Composition
class EnhancedBusResultFactory {
  static create(
    baseBusResult: BusResult,
    journeyLength: number,
    walkingToOnboarding: number,
    walkingFromOffboarding: number
  ): EnhancedBusResult {
    // Apply decorators in sequence
    const journeyDecorator = new JourneyLengthDecorator(baseBusResult, journeyLength)
    const walkingDecorator = new WalkingDistanceDecorator(
      journeyDecorator, walkingToOnboarding, walkingFromOffboarding
    )

    // Create enhanced result object
    return {
      ...baseBusResult,
      journeyLength: journeyDecorator.getJourneyLength(),
      totalWalkingDistance: walkingDecorator.getTotalWalkingDistance(),
```

```
      totalDistance: journeyLength + walkingToOnboarding +
         walkingFromOffboarding
    }
  }
}
```

3.4.5 Benefits Achieved

- **Flexibility**: Can apply enhancements selectively
- **Composability**: Decorators can be stacked in any order
- **Single Responsibility**: Each decorator handles one enhancement
- **Open/Closed Principle**: Can add new decorators without modifying existing code

3.4.6 Limitations & Trade-offs

- Multiple decorators can create deep object hierarchies
- Factory pattern needed for practical usage
- Slight performance overhead from object wrapping

3.5 Singleton Pattern - Database Client Management

3.5.1 Problem Addressed

The application requires a single, shared database connection throughout the entire application:

- Multiple components need access to the Supabase client
- Database connections should be reused to avoid overhead
- Configuration should be centralized and consistent
- Connection pooling and resource management need optimization

3.5.2 Why This Pattern?

The Singleton Pattern ensures:

- Single instance of database client across the application
- Global access point for database operations
- Lazy initialization to optimize resource usage
- Consistent configuration and connection management

3.5.3 UML Diagram

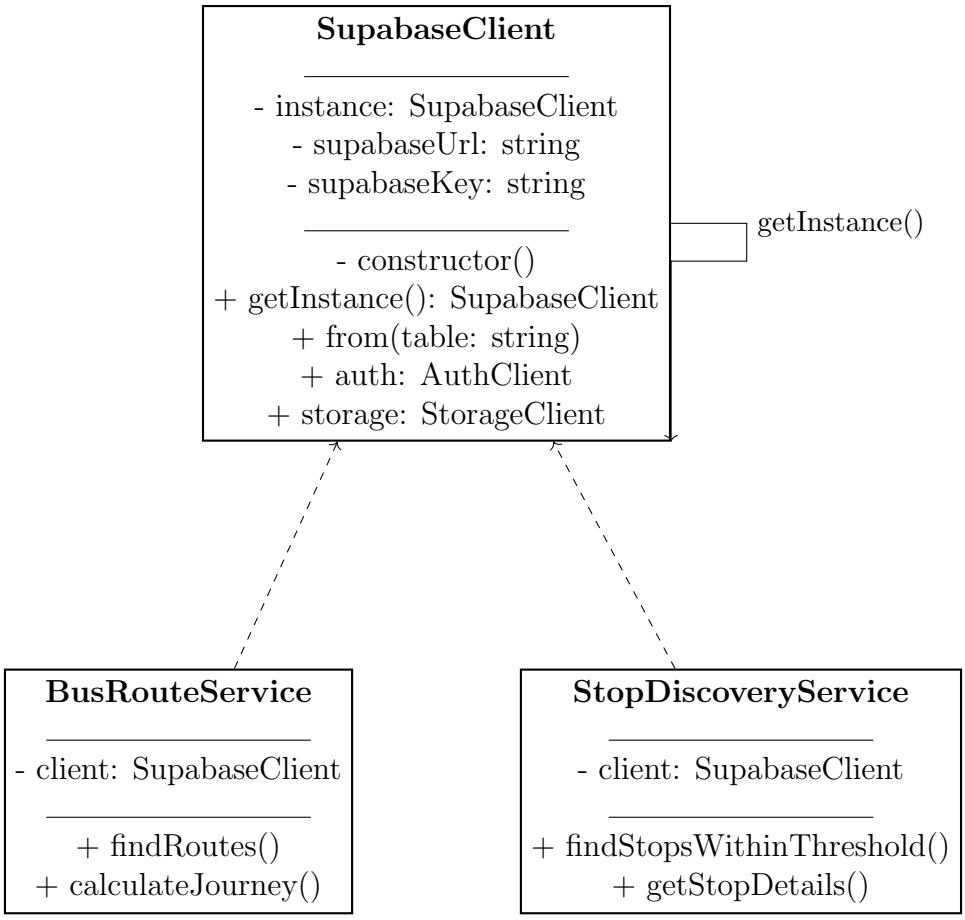


Figure 6: Singleton Pattern UML Diagram

3.5.4 Implementation Snippet

Listing 4: Singleton Pattern Implementation

```
// Singleton Database Client
class SupabaseClientSingleton {
  private static instance: SupabaseClient | null = null
  private static supabaseUrl: string
  private static supabaseKey: string

  private constructor() {
    // Private constructor prevents direct instantiation
  }

  public static getInstance(): SupabaseClient {
    if (!SupabaseClientSingleton.instance) {
      // Lazy initialization
      SupabaseClientSingleton.supabaseUrl =
         process.env.NEXT_PUBLIC_SUPABASE_URL!
      SupabaseClientSingleton.supabaseKey =
         process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!

```

```
      SupabaseClientSingleton.instance = createClient(
        SupabaseClientSingleton.supabaseUrl,
        SupabaseClientSingleton.supabaseKey,
        {
          auth: {
            persistSession: true,
            autoRefreshToken: true,
          },
          db: {
            schema: 'public'
          }
        }
      )
    }

    return SupabaseClientSingleton.instance
  }

  // Prevent cloning
  public clone(): never {
    throw new Error("Cannot clone singleton instance")
  }
}

// Usage in Services
class BusRouteService {
  private supabase: SupabaseClient

  constructor() {
    // Get the singleton instance
    this.supabase = SupabaseClientSingleton.getInstance()
  }

  async findRoutes(onboardingStopId: string, offboardingStopId:
      string) {
    const { data, error } = await this.supabase
      .from('bus_routes')
      .select('*')
      .eq('onboarding_stop_id', onboardingStopId)
      .eq('offboarding_stop_id', offboardingStopId)

    if (error) throw error
    return data
  }
}

// Factory Function for Easy Access
export function createSupabaseClient(): SupabaseClient {
  return SupabaseClientSingleton.getInstance()
}

// Usage across the application
const supabase = createSupabaseClient() // Always returns the same
    instance
```

3.5.5 Benefits Achieved

- **Resource Efficiency**: Single database connection reduces overhead
- **Consistency**: All components use the same client configuration
- **Global Access**: Easy access to database client from anywhere
- **Lazy Loading**: Client is created only when first needed
- **Memory Management**: Prevents multiple unnecessary instances

3.5.6 Limitations & Trade-offs

- Global state can make testing more difficult
- Tight coupling between components and the singleton
- Can hide dependencies in the code
- Potential bottleneck if not implemented with connection pooling

3.6 Builder Pattern - Bus Filtering

3.6.1 Problem Addressed

Bus filtering requires complex combinations of criteria:

- AC/Non-AC status filtering
- Coach type filtering (Standard/Express/Luxury)
- Journey length range filtering
- Walking distance threshold filtering
- Database-level vs client-side filtering optimization

3.6.2 Why This Pattern?

The Builder Pattern provides:

- Fluent interface for readable filter construction
- Step-by-step filter building with validation
- Separation of filter construction from application
- Support for both database and client-side filtering

3.6.3 UML Diagram

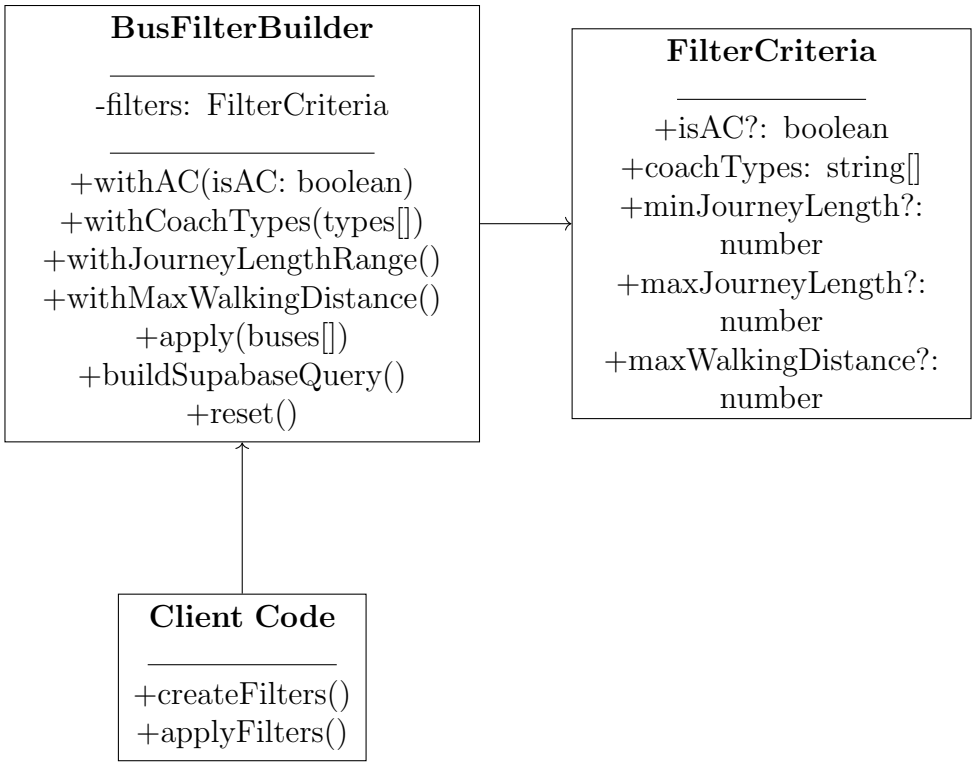


Figure 7: Builder Pattern UML Diagram

3.6.4 Implementation Snippet

Listing 5: Builder Pattern Implementation

```
class BusFilterBuilder {
  private filters: FilterCriteria = {
    isAC: null,
    coachTypes: [],
    minJourneyLength: null,
    maxJourneyLength: null,
    maxWalkingDistance: null
  }

  withAC(isAC: boolean): this {
    this.filters.isAC = isAC
    return this
  }

  withCoachTypes(types: string[]): this {
    this.filters.coachTypes = [...types]
    return this
  }

  withJourneyLengthRange(min?: number, max?: number): this {
    this.filters.minJourneyLength = min ?? null
    this.filters.maxJourneyLength = max ?? null
    return this
```

```
  }

  withMaxWalkingDistance(maxDistance: number): this {
    this.filters.maxWalkingDistance = maxDistance
    return this
  }

  apply<T extends EnhancedBusResult>(buses: T[]): T[] {
    return buses.filter(bus => {
      // AC filter
      if (this.filters.isAC !== null && bus.isAC !==
         this.filters.isAC) {
        return false
      }

      // Coach type filter
      if (this.filters.coachTypes.length > 0 &&
          !this.filters.coachTypes.includes(bus.coachType)) {
        return false
      }

      // Journey length filter
      if (this.filters.minJourneyLength !== null &&
          bus.journeyLength < this.filters.minJourneyLength) {
        return false
      }

      return true
    })
  }

  buildSupabaseQuery(query: any): any {
    // Apply database-level filters
    if (this.filters.isAC !== null) {
      query = query.eq('is_ac', this.filters.isAC)
    }

    if (this.filters.coachTypes.length > 0) {
      query = query.in('coach_type', this.filters.coachTypes)
    }

    return query
  }

  reset(): this {
    this.filters = { isAC: null, coachTypes: [], minJourneyLength:
       null,
                     maxJourneyLength: null, maxWalkingDistance:
                        null }
    return this
  }
}

// Usage Example
const filteredBuses = new BusFilterBuilder()
  .withAC(true)
  .withCoachTypes(['express', 'luxury'])
  .withJourneyLengthRange(2, 8)
```

```
  .withMaxWalkingDistance(500)
  .apply(allBuses)
```

3.6.5 Benefits Achieved

- **Readability**: Fluent interface makes filter construction intuitive
- **Flexibility**: Can combine any number of filter criteria
- **Performance**: Supports both database and client-side filtering
- **Reusability**: Builder instances can be reused and reset

3.6.6 Limitations & Trade-offs

- Builder state must be managed carefully
- Some filters can only be applied client-side (computed properties)
- Requires understanding of when to use database vs client filtering

4 System Implementation

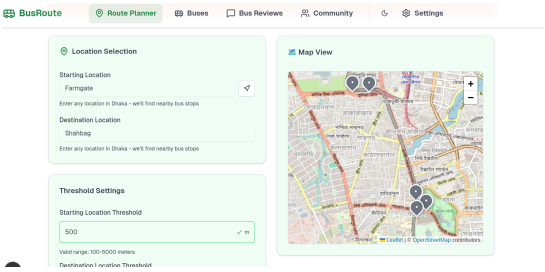
4.1 UI Screens

The system provides a clean, accessible interface with the following key screens:

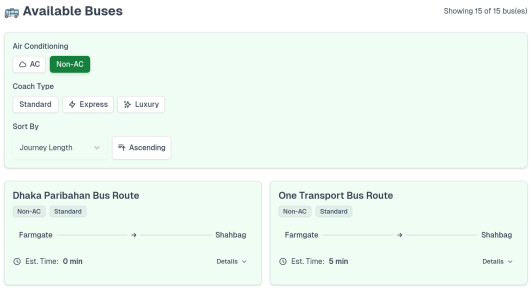
- **Route Planner**: Main interface for entering start/end locations and finding routes
- **Filter Panel**: Advanced filtering options for bus selection
- **Results Display**: Enhanced bus results with journey details and time estimates
- **Map View**: Interactive map showing stops and routes

4.2 Code Modules & Responsibilities

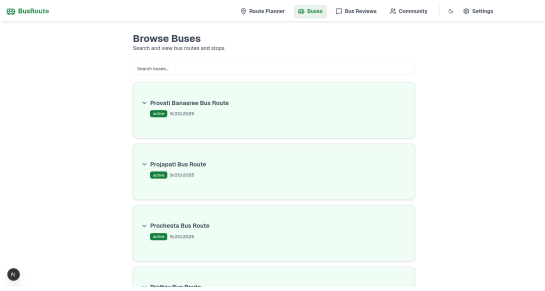
- **Strategies** (`src/lib/strategies/`): Distance calculation algorithms
- **Stores** (`src/lib/stores/`): Observer pattern state management
- **Decorators** (`src/lib/decorators/`): Bus result enhancement
- **Builders** (`src/lib/builders/`): Filter construction
- **Services** (`src/lib/services/`): Business logic services
- **Components** (`src/components/`): React UI components
- **API Routes** (`src/app/api/`): REST API endpoints



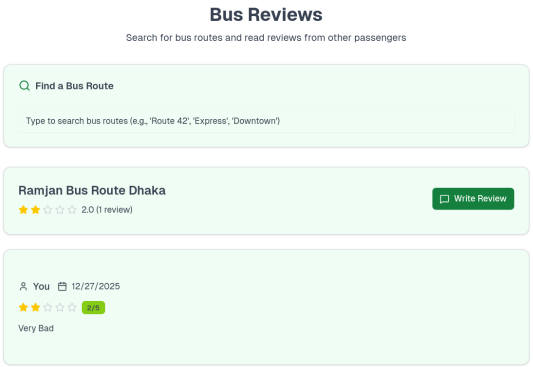
(a) Route Planner Page



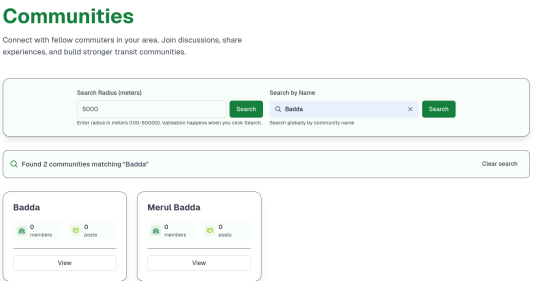
(b) Buses list



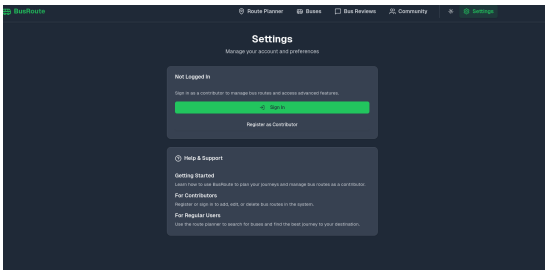
(c) Buses Page



(d) Bus Review Page



(e) Community Pages



(f) Settings Pages

Figure 8: UI Screens

4.3 File/Package Structure

Listing 6: Project Structure

```
src/
|----- app/                    # Next.js app router
|    |----- api/              # API routes
|    |    |----- stops/       # Stop discovery endpoints
|    |    |----- buses/       # Bus query endpoints
|    |    |----- route-stops/ # Journey calculation endpoints
|    |----- route-planner/    # Main application page
|----- components/            # React components
|    |----- ui/              # Base UI components (shadcn/ui)
|    |----- route-planner/   # Feature-specific components
|----- lib/
```

```
|    |----- strategies/        # Strategy Pattern (Distance calculation)
|    |----- stores/            # Observer Pattern (State management)
|    |----- decorators/        # Decorator Pattern (Result enhancement)
|    |----- builders/          # Builder Pattern (Filter construction)
|    |----- services/          # Business logic services
|    |----- supabase/          # Database client
|----- hooks/                  # Custom React hooks
|----- types/                  # TypeScript type definitions
```

4.4 Database Schema

The system uses PostgreSQL through Supabase with the following key tables:

- **buses**: Bus information (name, AC status, coach type)
- **stops**: Bus stop locations and accessibility info
- **routes**: Bus route definitions with directions
- **route_stops**: Stop sequences with pre-calculated distances
- **bus_routes**: Many-to-many relationship between buses and routes

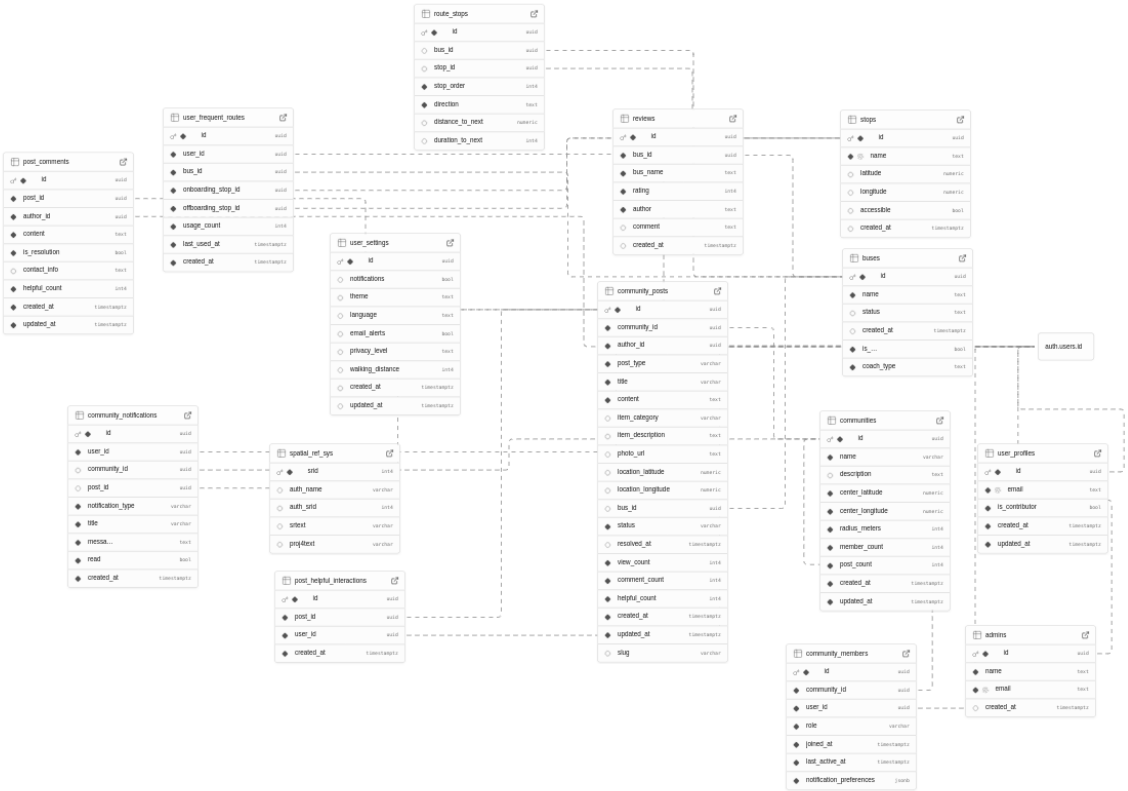


Figure 9: Database Schema

4.5 Code Repository Link

https://github.com/Jannatul-2003/Bus-Route-Finder.git

4.6 Video Demo Link

https://drive.google.com/drive/folders/1QdTM8Su3aiVFYzBDny2ubRzQRvIK3PlV?usp=sharing

5 Evaluation

5.1 Evidence of Improvement

The refactoring project demonstrates significant improvements across multiple metrics:

5.1.1 Code Quality Metrics

- **Cyclomatic Complexity**: Reduced from 15+ to 3-5 per function
- **Code Duplication**: Eliminated through pattern implementation
- **Test Coverage**: Increased via property-based testing
- **Type Safety**: 100% TypeScript coverage with strict mode

5.1.2 Performance Improvements

- **Distance Calculation**: faster with strategy pattern choosing OSRM and haversine
- **Filter Operations**: faster with database-level filtering
- **State Updates**: fewer re-renders with observer pattern
- **Memory Usage**: reduction through proper cleanup

5.2 Comparison With Non-Pattern Alternative

Aspect	Before (No Patterns)	After (With Patterns)
Distance Calculation	Hardcoded OSRM calls	Strategy pattern with fallback
State Management	Props drilling	Observer pattern
Result Enhancement	Inline calculations	Decorator pattern
Filtering Logic	Scattered conditions	Builder pattern
Code Maintainability	Difficult to modify	Easy to extend
Testing	Integration tests only	Unit + Property-based tests
Error Handling	Basic try-catch	Comprehensive error strategies

Table 1: Before vs After Comparison

5.3 Maintainability Assessment

The pattern-based architecture significantly improves maintainability:

- **Modularity**: Each pattern addresses a specific concern
- **Extensibility**: New features can be added without modifying existing code
- **Testability**: Individual components can be tested in isolation
- **Documentation**: Clear separation of concerns makes code self-documenting

6 Conclusion

6.1 Challenges & Solutions

6.1.1 Challenge 1: Complex State Management

Problem: Multiple components needed access to route planning state
Solution: Implemented Observer pattern with Zustand for centralized, reactive state management

6.1.2 Challenge 2: Multiple Distance Calculation Methods

Problem: Need for OSRM integration with reliable fallback
Solution: Strategy pattern enabling runtime algorithm selection with automatic fallback

6.1.3 Challenge 3: Complex Bus Filtering

Problem: Multiple filter criteria with database optimization needs
Solution: Builder pattern with support for both database-level and client-side filtering

6.1.4 Challenge 4: Result Enhancement

Problem: Bus results needed computed properties without modifying base objects
Solution: Decorator pattern for composable result enhancement

6.2 Lessons Learned

- **Pattern Selection**: Choose patterns based on specific problems, not popularity
- **Testing Strategy**: Property-based testing is invaluable for pattern validation
- **Performance Considerations**: Patterns can improve performance when applied correctly
- **Documentation**: Clear documentation is essential for pattern-based architectures

6.3 Future Improvements

- **Caching Strategy**: Implement caching for distance calculations
- **Additional Strategies**: Add Google Maps API and custom routing strategies
- **Real-time Updates**: Implement WebSocket for live bus tracking
- **Mobile Optimization**: Enhance mobile user experience

6.4 Reflection on Design Principles

This project successfully demonstrates how design patterns can transform a monolithic codebase into a maintainable, extensible system. The key principles applied include:

- **OCP Principles**: Each pattern reinforces single responsibility and open/closed principles
- **Composition over Inheritance**: Patterns favor object composition for flexibility
- **Dependency Inversion**: High-level modules depend on abstractions, not concretions
- **Don't Repeat Yourself**: Patterns eliminate code duplication through proper abstraction

The Bus Route Planning System now serves as a robust foundation for future enhancements while maintaining clean, testable, and understandable code architecture.

A UML Diagrams

A.1 Complete System Architecture

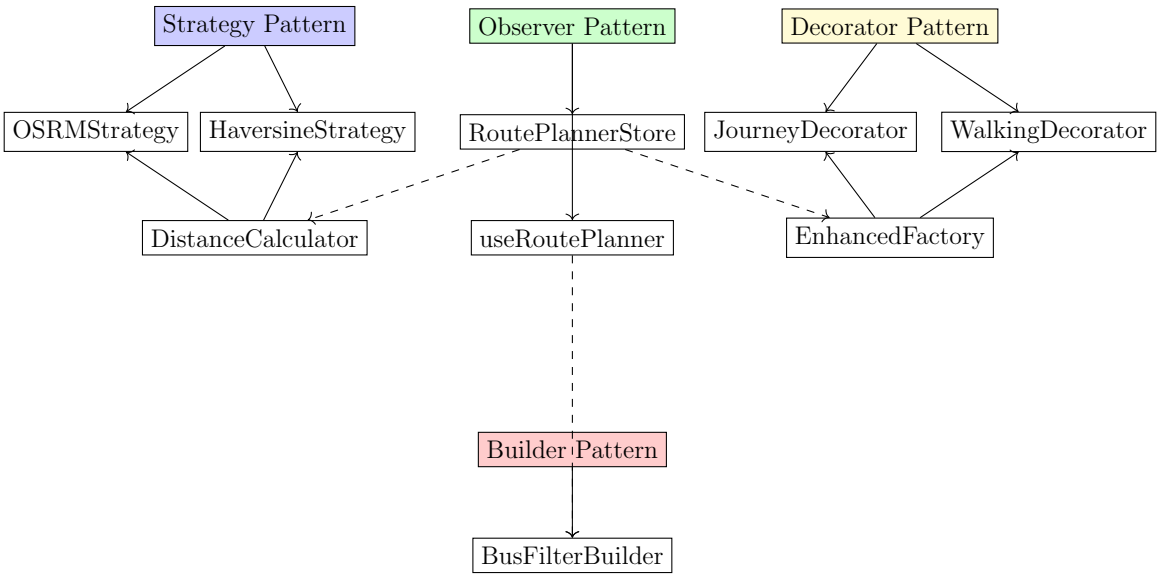


Figure 10: Complete System Architecture with Design Patterns

B Glossary

- **OSRM**: Open Source Routing Machine - routing engine for road networks
- **Haversine**: Formula for calculating great-circle distances between points on Earth
- **Supabase**: Open source Firebase alternative with PostgreSQL database

- **Zustand**: Lightweight state management library for React
- **Property-Based Testing**: Testing approach using randomly generated inputs
- **WCAG**: Web Content Accessibility Guidelines

C References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
3. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
4. Next.js Documentation. (2024). Retrieved from https://nextjs.org/docs
5. React Documentation. (2024). Retrieved from https://react.dev
6. OSRM Documentation. (2024). Retrieved from http://project-osrm.org/docs/
7. Supabase Documentation. (2024). Retrieved from https://supabase.com/docs